#2 We insert a new entry in the treap: the key will be the method's argument, while the priority will be a random real number.

3.4.3 Applications of Randomized Treaps

As we have seen in the last section, randomized treaps are the main application for treaps; now the question is, what are the most common applications for randomized treaps?

In general, we can use a randomized treap everywhere we would use a BST, and in particular they are indicated for situations where we need a balanced tree, but we are fine with having guarantees on the average case only, not on the worst-case.

Another aspect to consider is that, as always when randomization and "average case" bounds are involved, the guarantees are more likely to hold when the number of entries is larger, while for smaller trees, it's easier to obtain skewed structures (for small trees, however, the performance difference between slightly skewed and balanced trees will also be less relevant, obviously).

BSTs, in turn, are often used to implement dictionaries and sets (we will see more about these structures in chapter 4).

Other examples include keeping data read from a stream sorted, counting the number of elements smaller (larger) than any given element of a dynamic set, and in general all applications where we need to keep a dynamic set of elements in sorted order, while supporting fast search, insertion and removal.

Practical examples of real-world code using BSTs are, for instance, managing a set of virtual memory areas (VMAs) in operating system kernels and keeping track of packet IP's verification ID.